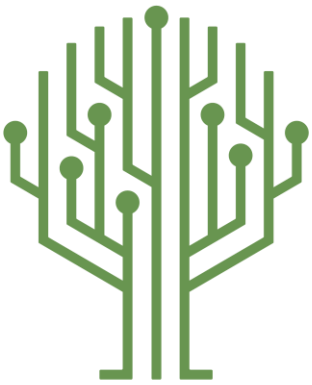


Green Pace Secure Development Policy

Contents

Overview 2
Purpose 2
Scope 2
Module Three Milestone 2
Ten Core Security Principles 2
C/C++ Ten Coding Standards 3
Coding Standard 1 4
Coding Standard 2 5
Coding Standard 3 6
Coding Standard 4 7
Coding Standard 5 8
Coding Standard 6 9
Coding Standard 7 10
Coding Standard 8 11
Coding Standard 9 13
Coding Standard 10 14
Defense-in-Depth Illustration 15
Project One 15
1. Revise the C/C++ Standards 15
2. Risk Assessment 15
3. Automated Detection 15
4. Automation 15
5. Summary of Risk Assessments 16
6. Create Policies for Encryption and Triple A 16
7. Map the Principles 17
Audit Controls and Management 18
Enforcement 18
Exceptions Process 18
Distribution 19
Policy Change Control 19
Policy Version History 19
Appendix A Lookups 19
Approved C/C++ Language Acronyms 19

Overview

Software development at Green Pace requires consistent implementation of secure principles to all developed applications. Consistent approaches and methodologies must be maintained through all policies that are uniformly defined, implemented, governed, and maintained over time.

Purpose

This policy defines the core security principles; C/C++ coding standards; authorization, authentication, and auditing standards; and data encryption standards. This article explains the differences between policy, standards, principles, and practices (guidelines and procedure): Understanding the Hierarchy of Principles, Policies, Standards, Procedures, and Guidelines.

Scope

This document applies to all staff that create, deploy, or support custom software at Green Pace.

Module Three Milestone

Ten Core Security Principles

Principles	Write a short paragraph explaining each of the 10 principles of security.
1. Validate Input Data	All data coming from users, files, networks, or other outside sources should be checked before the program uses it. This helps prevent invalid or dangerous data from causing errors or security problems. CERT identifies input validation as an important way to prevent software vulnerabilities.
2. Heed Compiler Warnings	Developers should pay attention to compiler warnings and fix them instead of ignoring them. Warnings can point out mistakes that may cause bugs or security problems. CERT recommends using the highest useful warning level and correcting the problems identified by the compiler.
3. Architect and Design for Security Policies	Security should be considered when a program is designed, not added only after the program is finished. The design should include security controls such as access restrictions and separation of important functions.
4. Keep It Simple	Software should be designed and written as simply as possible. Simple code is easier to understand, test, maintain, and check for security problems.
5. Default Deny	A system should deny access unless permission has specifically been given. This prevents users or programs from automatically receiving access to information or functions they should not have
6. Adhere to the Principle of Least Privilege	Users and programs should only receive the permissions they need to perform their jobs. Limiting permissions reduces the damage that can occur if an account or program is compromised.
7. Sanitize Data Sent to Other Systems	Data sent to databases, operating systems, or other external systems should be checked and safely handled first. Special characters or unexpected data can

Principles	Write a short paragraph explaining each of the 10 principles of security.
	sometimes change how another system interprets the information. CERT specifically discusses sanitizing data before sending it to complex subsystems
8. Practice Defense in Depth	Security should use multiple layers instead of depending on one security control. If one security measure fails, another layer can still help protect the system and its data.
9. Use Effective Quality Assurance Techniques	Software should be tested and reviewed throughout development. Testing, code reviews, and security analysis can find problems before software is released.
10. Adopt a Secure Coding Standard	Developers should follow a consistent set of secure coding rules. Using a standard such as the SEI CERT C++ Coding Standard gives developers specific practices to follow when writing and reviewing code. CERT organizes its rules by areas such as declarations, expressions, strings, memory, and error handling.

C/C++ Ten Coding Standards

Complete the coding standards portion of the template according to the Module Three milestone requirements. In Project One, follow the instructions to add a layer of security to the existing coding standards. Please start each standard on a new page, as they may take up more than one page. The first seven coding standards are labeled by category. The last three are blank so you may choose three additional standards. Be sure to label them by category and give them a sequential number for that category. Add compliant and noncompliant sections as needed to each coding standard.

Green Pace

Coding Standard 1

Coding Standard	Label	Name of Standard
Data Type	[STD-001-CPP]	Developers must select appropriate data types and make sure values fit within the limits of those types. Incorrect types or unexpected values can cause calculation errors and security vulnerabilities. Developers should follow the SEI CERT C++ rule INT31-CPP when converting integers to prevent loss of data or unexpected results.

Noncompliant Code

This example uses an `int` to store a value that should not be negative, but it does not properly account for the value's range.

[Noncompliant code block; code should be indented using 12-point Courier New font.]

```
int age = -5;

if (age >= 0) {

          std::cout << "Valid age";

}
```

Compliant Code

This example validates the value before using it. The program only accepts an age within a reasonable range.

```
int age = 25;

if (age >= 0 && age <= 120) {

       std::cout << "Valid age";

}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Green Pace

Principles(s): Principle 1: Validate Input Data. Checking input values helps ensure they are valid before the program uses them.

Principle 2: Heed Compiler Warnings. Compiler warnings can identify unsafe conversions and possible data type problems.

Principle 4: Keep It Simple. Clear data types and straightforward validation make code easier to understand and review.

Principle 9: Use Effective Quality Assurance Techniques. Code reviews and automated checks can help find data type problems before software is released.

Principle 10: Adopt a Secure Coding Standard. Following consistent C++ rules helps developers handle data types safely across applications.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Unlikely	Medium	2	High

Automation

Tool	Version	Checker	Description Tool
Clang-Tidy	18.1.8	bugprone-narrowing-conversions	Detects conversions that may lose data or produce unexpected results.
GCC	14.2	Wconversion	Warns about potentially unsafe conversions between data types.
Clang	18.1.8	Wconversion	Identifies conversions that may change a value unexpectedly.
CodeQL	2.20.4	C/C++ code analysis	Can identify certain data flow and type-related weaknesses when supported by the configured queries.

Coding Standard 2

Coding Standard	Label	Name of Standard
Data Value	[STD-002-CPP]	Programs should check that values are within acceptable ranges before using them. This prevents unexpected values from causing incorrect calculations or unsafe program behavior.

Noncompliant Code

This code assumes that the value entered by the user is valid without checking it first.

```
int quantity;


std::cin >> quantity;


int total = quantity * 100;


std::cout << total;
```

Compliant Code

This version checks that the quantity is positive before using it in the calculation.

```
int quantity;


std::cin >> quantity;


if (quantity > 0) {
    int total = quantity * 100;
```

Compliant Code

```
    std::cout << total;

}

else {

    std::cout << "Invalid quantity";

}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 1: Validate Input Data. Programs should check values received from users or outside sources before using them.

Principle 2: Heed Compiler Warnings. Compiler warnings can help identify calculations and operations that may produce unexpected results.

Principle 4: Keep It Simple. Clear validation checks make it easier to understand which values the program accepts.

Principle 9: Use Effective Quality Assurance Techniques. Testing different values, including invalid and unusually large values, helps identify problems before release.

Principle 10: Adopt a Secure Coding Standard. Following consistent validation rules helps prevent unsafe values from affecting the application.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	4	High

Automation

Tool	Version	Checker	Description Tool
Clang-Tidy	18.1.8	bugprone-integer-division	Identifies integer division that may produce unexpected results.
GCC	14.2	Wall -Wextra	Reports common programming

Tool	Version	Checker	Description Tool
			mistakes and suspicious code that may affect calculations.
Clang	18.1.8	-fsanitize=undefined	Detects certain runtime problems, including signed integer overflow, during testing.
CodeQL	2.20.4	C/C++ code analysis	Can identify certain unsafe data flows and calculation-related weaknesses using supported queries.

Coding Standard 3

Coding Standard	Label	Name of Standard
String Correctness	[STD-003-CPP]	Strings must have enough storage for the characters they contain, including the null terminator when using C style strings. CERT specifically recommends guaranteeing enough storage for string data and the null terminator.

Noncompliant Code

The character array is too small for the string being copied into it. This can cause a buffer overflow.

```
char name[5];


strcpy(name, "Jose");
```

Compliant Code

The array is large enough to hold the characters and the null terminator. Using `std::string` is another safer option for many situations.

```
std::string name = "Jose";


std::cout << name;
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 1: Validate Input Data. Check strings received from users or outside sources before using them.

Principle 4: Keep It Simple. Using safer string-handling methods makes code easier to understand and maintain.

Principle 7: Sanitize Data Sent to Other Systems. String data should be handled safely before it is passed to databases or other systems.

Principle 8: Practice Defense in Depth. Safe string handling should be combined with testing and automated security checks.

Principle 9: Use Effective Quality Assurance Techniques. Code reviews and testing can identify buffer overflows and string-handling errors.

Principle 10: Adopt a Secure Coding Standard. Following secure string-handling rules helps developers avoid common vulnerabilities.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	4	High

Automation

Tool	Version	Checker	Description Tool
Clang	18.1.8	AddressSanitizer (`-fsanitize=address`)	Detects many buffer overflows and memory access errors during testing.
GCC	14.2	-Wstringop-overflow	Warns about certain operations that may write beyond a string buffer.
Clang-Tidy	18.1.8	bugprone-not-null-terminated-result	Identifies certain string operations that may produce results without proper null termination.
CodeQL	2.20.4	C/C++ code analysis	Can identify certain buffer and data-flow vulnerabilities using supported queries.

Coding Standard 4

Coding Standard	Label	Name of Standard
SQL Injection	[STD-004-CPP]	SQL queries should not be created by directly combining untrusted user input with SQL commands. SQL injection can allow an attacker to change the intended query and potentially access or modify information. CERT identifies parameterized queries along with validating and sanitizing untrusted input, as primary protections against SQL injection.

Noncompliant Code

This example places user input directly into a SQL statement. An attacker could enter SQL commands instead of a normal username.

```
std::string username;


std::cin >> username;


std::string query = "SELECT * FROM USERS WHERE NAME='" + username + "'";
```

Compliant Code

The application should use a parameterized SQL statement so that user input is treated as data instead of part of the SQL command.

```
sqlite3_stmt* stmt;


sqlite3_prepare_v2(
    db,
    "SELECT * FROM USERS WHERE NAME = ?",
    -1,
    &stmt,
    nullptr
```

Compliant Code

```
);



sqlite3_bind_text(

    stmt,

  1,
  username.c_str(),
  -1,
  SQLITE_TRANSIENT
);
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 1: Validate Input Data. Check user input before using it in database operations.

Principle 5: Default Deny. Database access should be denied unless it is specifically authorized.

Principle 6: Adhere to the Principle of Least Privilege. Database accounts should only have the permissions required for their tasks.

Principle 7: Sanitize Data Sent to Other Systems. Use parameterized queries so input cannot change the intended SQL command.

Principle 8: Practice Defense in Depth. Combine parameterized queries with access controls, logging, and security testing.

Principle 9: Use Effective Quality Assurance Techniques. Automated security checks and testing can help identify unsafe database operations.

Principle 10: Adopt a Secure Coding Standard. Developers should consistently follow secure database coding practices.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	High	low	5	High

Automation

Tool	Version	Checker	Description Tool

Tool	Version	Checker	Description Tool
CodeQL	2.20.4	C/C++ SQL injection queries	Can identify potential SQL injection paths when supported by the configured queries and data-flow models.
SonarQube Server	2025.1	C/C++ security analysis	Can flag certain security weaknesses in supported languages and configurations.
Clang-Tidy	18.1.8	C++ code analysis checks	Helps detect selected code-quality problems, but does not independently detect every SQL injection vulnerability.
Manual security testing	Not applicable	SQL injection test cases	Tests whether unexpected input can change database query behavior.

Coding Standard 5

Coding Standard	Label	Name of Standard
Memory Protection	[STD-005-CPP]	Programs must properly manage memory and avoid using memory after it has been released. CERT's `MEM50-CPP` specifically states that programs should not access free memory.

Noncompliant Code

The program deletes dynamically allocated memory and then tries to use the pointer again. This creates a use after a free problem.

```
int* number = new int(10);


delete number;


std::cout << *number;
```

Compliant Code

The program uses the memory before releasing it and then sets the pointer to `nullptr` after deletion.

```
int* number = new int(10);


std::cout << *number;


delete number;

number = nullptr;
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 2: Heed Compiler Warnings. Compiler warnings can help identify pointer misuse and possible memory problems.

Principle 4: Keep It Simple. Using safer memory-management methods makes code easier to maintain and less likely to contain errors.

Principle 8: Practice Defense in Depth. Memory safety should be supported by automated testing, code reviews, and other security controls.

Principle 9: Use Effective Quality Assurance Techniques. Testing tools can detect memory errors before software is released.

Principle 10: Adopt a Secure Coding Standard. Following consistent memory-management rules helps prevent use-after-free and related vulnerabilities.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	4	high

Automation

Tool	Version	Checker	Description Tool
Clang	18.1.8	AddressSanitizer (`-fsanitize=address`)	Detects many use-after-free, buffer overflow, and other memory access errors during testing.
Valgrind Memcheck	3.24.0	Memcheck	Detects invalid memory accesses, use of uninitialized values, and memory leaks.
Clang-Tidy	18.1.8	cppcoreguidelines-owning-memory	Flags certain uses of raw pointers that represent ownership of allocated memory.
GCC	14.2	Wall -Wextra	Reports common programming problems, including some suspicious pointer related code.

Coding Standard 6

Coding Standard	Label	Name of Standard
Assertions	[STD-006-CPP]	Assertions should be used to check assumptions made by the programmer during development. They should not replace normal runtime checks for user input, network errors, memory errors, or other conditions that can occur during normal operation. CERT specifically recommends using assertions for programmer assumptions rather than runtime error handling.

Noncompliant Code

This example uses an assertion to validate user input. Assertions may be disabled in a production build, so they should not be used as the only protection for user-controlled data.

```
int age;


std::cin >> age;


assert(age >= 18);
```

Compliant Code

The program uses a normal `if` statement to check user input. This check remains active when the program is deployed.

```
int age;


std::cin >> age;


if (age < 18) {

    std::cout << "Access denied";

}

else {
```

Compliant Code

```
    std::cout << "Access granted";

}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 2: Heed Compiler Warnings. Compiler warnings and other development checks can help identify incorrect assumptions and potential programming mistakes.

Principle 4: Keep It Simple. Clear assertions make it easier to understand the conditions that should always be true in the program.

Principle 9: Use Effective Quality Assurance Techniques. Testing and debugging help developers find problems with program assumptions before release.

Principle 10: Adopt a Secure Coding Standard. Consistent rules help developers use assertions correctly and avoid relying on them for security-critical input validation.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Possible	Low	3	Medium

Automation

Tool	Version	Checker	Description Tool
Clang-Tidy	18.1.8	bugprone-assert-side-effect	Identifies expressions with side effects inside assertions, which may behave differently when assertions are disabled.
GCC	14.2	-Wall -Wextra	Detects various common coding problems it does not catch every misuse of assertions.
Clang Static Analyzer	18.1.8	Static analysis checks	Examines code paths for certain potential bugs and incorrect assumptions.
Code review	Not	Assertion and validation review	Checks that user input and

Tool	Version	Checker	Description Tool
	applicable		runtime errors are handled with normal validation instead of relying only on assertions.

Coding Standard 7

Coding Standard	Label	Name of Standard
Exceptions	[STD-007-CPP]	Programs should properly handle exceptions instead of allowing unexpected errors to terminate the application. CERT includes exception-handling rules such as catching exceptions by reference and handling exceptions appropriately.

Noncompliant Code

This example throws an exception but does not provide a handler for it. The unhandled exception can cause the program to terminate.

```
#include <stdexcept>


void process() {
    throw std::runtime_error("Error");
}


int main() {
    process();
}
```

Compliant Code

The compliant version uses a try block and catches the exception so the program can respond to the error

```
#include <iostream>
#include <stdexcept>


void process() {
    throw std::runtime_error("Error");
```

Compliant Code

```
}


int main() {
    try {
        process();
    }
    catch (const std::exception& ex) {
        std::cout << "Error: " << ex.what();
    }
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 2: Heed Compiler Warnings. Compiler warnings can help identify problems with exception handling and unreachable code.

Principle 3: Architect and Design for Security Policies. Error handling should be planned as part of the application's design.

Principle 4: Keep It Simple. Clear exception-handling code makes errors easier to understand and manage.

Principle 8: Practice Defense in Depth. Exception handling should work alongside logging, validation, and other security controls.

Principle 9: Use Effective Quality Assurance Techniques. Testing error conditions helps confirm that the application responds safely when something goes wrong.

Principle 10: Adopt a Secure Coding Standard. Consistent error-handling rules help prevent crashes and reduce the risk of exposing sensitive information.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
Medium	Possible	Medium	3	Medium

Automation

Tool	Version	Checker	Description Tool
Clang-Tidy	18.1.8	bugprone-exception-escape	Identifies certain functions that may allow exceptions to escape unexpectedly.
Clang-Tidy	18.1.8	bugprone-throw-keyword-missing	Identifies certain suspicious exception handling patterns involving `throw`.
GCC	14.2	-Wall -Wextra	Detects common compiler warnings, including some issues involving exception related code.
Code review and testing	Not applicable	Exception-path testing	Checks that errors are handled safely, resources are released, and sensitive details are not exposed.

Coding Standard 8

Coding Standard	Label	Name of Standard
[Student Choice]	[STD-008-CPP]	Input from users and other outside sources should be checked before the program uses it. Input validation helps prevent unexpected or malicious values from reaching sensitive parts of the application. CERT lists input validation as one of its core secure coding practices.

Noncompliant Code

The program accepts user input and immediately uses it without checking whether it is valid

```
int age;


std::cin >> age;


std::cout << "Age: " << age;
```

Compliant Code

The program checks that the input operation succeeded and that the value is within an acceptable range

```
int age;


std::cin >> age;


if (std::cin.fail() || age < 0 || age > 120) {
    std::cout << "Invalid age";
}
else {
    std::cout << "Age: " << age;
```

Compliant Code

```
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 1: Validate Input Data. All external input should be checked before the program uses it.

Principle 3: Architect and Design for Security Policies. Input validation should be included in the application's design from the beginning.

Principle 5: Default Deny. Input that fails validation should be rejected instead of automatically accepted.

Principle 7: Sanitize Data Sent to Other Systems. Data should be safely handled before it is passed to databases or other systems.

Principle 8: Practice Defense in Depth. Input validation should be combined with other security controls, such as access restrictions and testing.

Principle 9: Use Effective Quality Assurance Techniques. Automated tests should include invalid, missing, and unexpected input.

Principle 10: Adopt a Secure Coding Standard. Consistent input-validation rules help prevent common security problems.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	low	4	High

Automation

Tool	Version	Checker	Description Tool
CodeQL	2.20.4	C/C++ data-flow analysis	Can identify certain cases where untrusted input reaches sensitive operations without appropriate checks.
Clang-Tidy	18.1.8	C++ static analysis checks	Identifies selected coding problems; it does not detect every missing input validation check.
GCC	14.2	-Wall -Wextra	Reports common programming issues that may affect input-

Tool	Version	Checker	Description Tool
			handling code.
Unit testing	Project-configured version	Input validation test cases	Tests valid, invalid, missing, and out-of-range values to confirm the program rejects unacceptable input.

Coding Standard 9

Coding Standard	Label	Name of Standard
[Student Choice]	[STD-009-CPP]	Variables should be initialized before they are used. Reading an uninitialized variable can result in unpredictable behavior and may expose incorrect or sensitive information. CERT has a C++ rule specifically addressing uninitialized memory.

Noncompliant Code

The variable `number` is declared but never given a value before it is used.

```
int number;

std::cout << number;
```

Compliant Code

The variable is initialized before it is used, giving the program a known starting value.

```
int number = 0;

std::cout << number;
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Principles(s):

Principle 2: Heed Compiler Warnings. Pay attention to compiler warnings about variables that may be uninitialized.

Principle 4: Keep It Simple. Use clear initialization so the program's behavior is easier to understand.

Principle 8: Defense in Depth. Use compiler checks, static analysis, and testing to catch mistakes.

Principle 9: Effective QA. Test code paths to ensure variables are initialized before use.

Principle 10: Adopt a Secure Coding Standard. Follow consistent rules for variable initialization.

Threat Level

Severity	Likelihood	Remediation Cost	Priority	Level

Green Pace

Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	low	4	High

Automation

Tool	Version	Checker	Description Tool
GCC	14.2	-Wuninitialized	The `-Wuninitialized` warning helps identify variables that may be used before they are initialized.
Clang	18.1.8	-Wuninitialized	The `-Wuninitialized` warning identifies potential uses of uninitialized variables.
Clang Static Analyzer	18.1.8	Detects possible uninitialized values	Analyzes program paths to detect situations where variables may be used before receiving a value.
CodeQL CLI	2.20.4	C/C++ static analysis for supported uninitialized-variable issues	Checks source code for possible uninitialized variables and other programming errors.

Coding Standard 10

Coding Standard	Label	Name of Standard
[Student Choice]	[STD-010-CPP]	Programs should make sure an array of indexes and container iterators remain within valid limits. Accessing data outside the valid range can cause crashes, incorrect results, or security vulnerabilities. CERT includes CTR50-CPP which requires container indexes and iterators to remain within valid ranges.

Noncompliant Code

The loop uses an index that can reach outside the valid range of the array.

```
int numbers[3] = {10, 20, 30};

for (int i = 0; i <= 3; i++) {
    std::cout << numbers[i] << std::endl;
}
```

Compliant Code

The loop uses < 3 instead of <= 3, so the largest index used is 2, which is inside the valid range of the array.

```
int numbers[3] = {10, 20, 30};

for (int i = 0; i < 3; i++) {
    std::cout << numbers[i] << std::endl;
}
```

Note: Stop here for the milestone. Complete this section for Project One in Module Six.

Green Pace

Principles(s):

Principle 1: Validate Input Data. Check indexes and sizes before accessing array elements.

Principle 2: Heed Compiler Warnings. Pay attention to warnings about possible out-of-bounds access.

Principle 4: Keep It Simple. Use clear checks to prevent invalid array or container access.

Principle 8: Defense in Depth. Combine bounds checks, static analysis, and testing.

Principle 9: Effective QA. Test normal, boundary, and invalid index values.

Principle 10: Adopt a Secure Coding Standard. Follow consistent rules for safe array and container access.

Threat Level

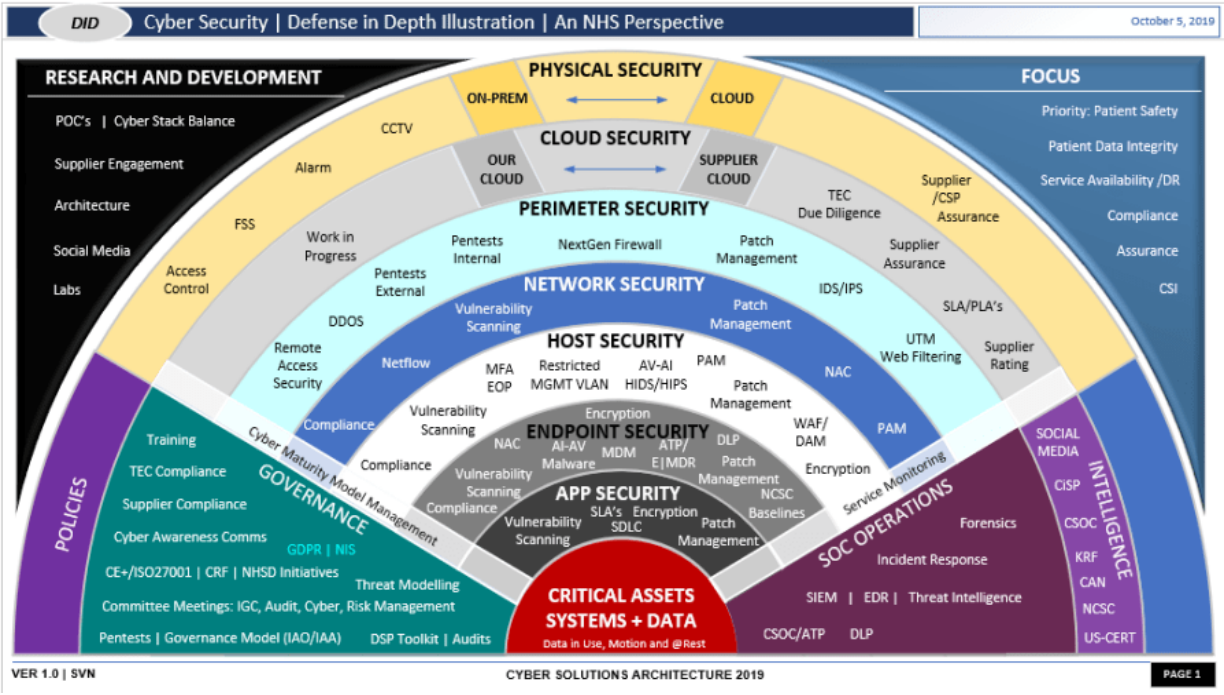
Severity	Likelihood	Remediation Cost	Priority	Level
High	Possible	Medium	4	High

Automation

Tool	Version	Checker	Description Tool
GCC	14.2	-Warray-bounds	Uses compiler warnings to identify code that may access an array outside its valid bounds.
Clang AddressSanitizer	18.1.8	-fsanitize=address	Detects out-of-bounds memory access errors while the program is running during testing.
Clang-Tidy	18.1.8	cppcoreguidelines-pro-bounds-constant-array-index	Analyzes C++ code for unsafe array indexing and pointer operations using available bounds-related checks.
Cppcheck	2.17.1	Array bounds and out-of-bounds access checks	Analyzes source code to identify possible array-bound errors and unsafe memory access.

Defense-in-Depth Illustration

This illustration provides a visual representation of the defense-in-depth best practice of layered security.



Project One

There are seven steps outlined below that align with the elements you will be graded on in the accompanying rubric. When you complete these steps, you will have finished the security policy.

Revise the C/C++ Standards

You completed one of these tables for each of your standards in the Module Three milestone. In Project One, add revisions to improve the explanation and examples as needed. Add rows to accommodate additional examples of compliant and noncompliant code. Coding standards begin on the security policy.

Risk Assessment

Complete this section on the coding standards tables. Enter high, medium, or low for each of the headers, then rate it overall using a scale from 1 to 5, 5 being the greatest threat. You will address each of the seven policy standards. Fill in the columns of severity, likelihood, remediation cost, priority, and level using the values provided in the appendix.

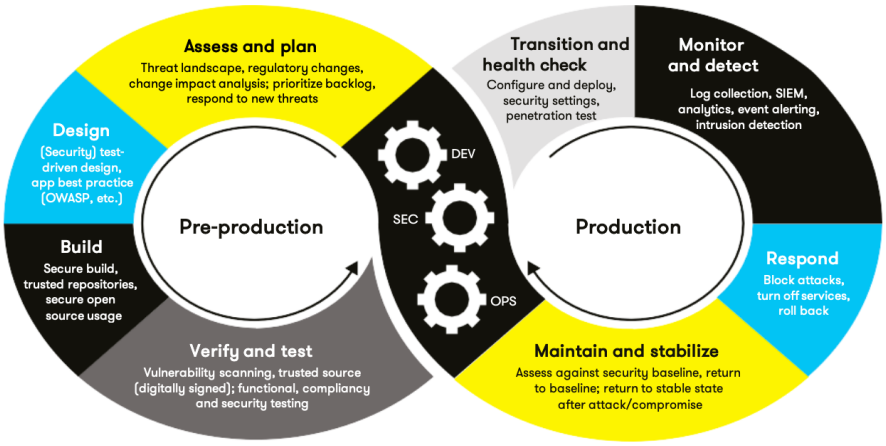
Automated Detection

Complete this section of each table on the coding standards to show the tools that may be used to detect issues. Provide the tool name, version, checker, and description. List one or more tools that can automatically detect this issue and its version number, name of the rule or check (preferably with link), and any relevant comments or description—if any. This table ties to a specific C++ coding standard.

Automation

Provide a written explanation using the image provided.

Green Pace



Automation will be used for the enforcement of and compliance to the standards defined in this policy. Green Pace already has a well-established DevOps process and infrastructure. Define guidance on where and how to modify the existing DevOps process to automate enforcement of the standards in this policy. Use the DevSecOps diagram and provide an explanation using that diagram as context.

Green Pace can use DevSecOps to check security throughout the software development process. As shown in the diagram, the process is a continuous cycle that helps the team find problems and make improvements.

First, the team should assess risks and plan how to protect the software. During the design and build stages, developers should follow the C/C++ security standards and use automated tools to check for coding mistakes. During testing, the team should scan security problems and fix them before releasing the software.

After release, Green Pace should monitor the software threats and respond to problems. If an issue is found, the team should fix it and use what they learned to improve security. Adding these steps to the existing DevOps process will help Green Pace find problems earlier and keep its software secure.

Summary of Risk Assessments

Consolidate all risk assessments into one table including both coding and systems standards, ordered by standard number.

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-001-CPP	High	Unlikely	Medium	High	2
STD-002-CPP	High	Possible	Medium	High	4
STD-003-CPP	High	Possible	Medium	High	4
STD-004-CPP	High	Possible	Low	High	5
STD-005-CPP	High	Possible	Medium	High	4

Rule	Severity	Likelihood	Remediation Cost	Priority	Level
STD-006-CPP	Medium	Possible	Low	Medium	3
STD-007-CPP	Medium	Possible	Medium	Medium	3
STD-008-CPP	High	Possible	Low	High	4
STD-009-CPP	High	Possible	Low	High	4
STD-010-CPP	High	Possible	Medium	High	4
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]

Create Policies for Encryption and Triple A

Include all three types of encryption (in flight, at rest, and in use) and each of the three elements of the Triple-A framework using the tables provided**.**

a. Explain each type of encryption, how it is used, and why and when the policy applies.
b. Explain each type of Triple-A framework strategy, how it is used, and why and when the policy applies.

Write policies for each and explain what it is, how it should be applied in practice, and why it should be used.

a. Encryption	**Explain what it is and how and why the policy applies.**
Encryption at rest	Green Pace must encrypt sensitive data stored in databases, files, and backups. Access to encryption keys should be limited to authorized users. This helps protect information if a device or storage system is stolen or accessed without permission.
Encryption in flight	Green Pace must protect data while it travels between users, applications, and servers. Secure connections such as TLS should be used to prevent attackers from reading or changing information during transfer. Outdated security protocols should not be used.
Encryption in use	Green Pace should protect sensitive data while it is being processed by an application. Access should be limited to authorized users and programs. Sensitive information should not be unnecessarily copied or saved in temporary files or logs.

b. Triple-A Framework*	**Explain what it is and how and why the policy applies.**
Authentication	Green Pace must verify the identity of users before allowing them to access its systems. Users should have their own accounts and strong passwords. Multi-factor

b. Triple-A Framework*	Explain what it is and how and why the policy applies.
	authentication should be used for administrators and other accounts that need extra protection. Failed login attempts should be recorded to help identify suspicious activity.
Authorization	Green Pace must make sure users can only access the information and systems they need for their jobs. Users should not automatically receive access to everything. Permissions should be reviewed regularly, and access should be removed when it is no longer needed.
Accounting	Green Pace must keep records of important activities, such as login attempts, changes to permissions, and access to sensitive information. These records can help the company investigate security problems. Logs should be protected from unauthorized changes, and passwords should never be included in them.

*****Use this checklist for the Triple A to be sure you include these elements in your policy:

- User logins
- Changes to the database
- Addition of new users
- User level of access
- Files accessed by users

Map the Principles

Map the principles to each of the standards, and provide a justification for the connection between the two. In the Module Three milestone, you added definitions for each of the 10 principles provided. Now it's time to connect the standards to principles to show how they are supported by principles. You may have more than one principle for each standard, and the principles may be used more than once. Principles are numbered 1 through 10. You will list the number or numbers that apply to each standard, then explain how each of these principles supports the standard. This exercise demonstrates that you have based your security policy on widely accepted principles. Linking principles to standards is a best practice.

NOTE: Green Pace has already successfully implemented the following:

- Operating system logs
- Firewall logs
- Anti-malware logs

The only item you must complete beyond this point is the Policy Version History table.

Audit Controls and Management

Every software development effort must be able to provide evidence of compliance for each software deployed into any Green Pace managed environment.

Evidence will include the following:

- Code compliance to standards
- Well-documented access-control strategies, with sampled evidence of compliance
- Well-documented data-control standards defining the expected security posture of data at rest, in flight, and in use
- Historical evidence of sustained practice (emails, logs, audits, meeting notes)

Enforcement

The office of the chief information security officer (OCISO) will enforce awareness and compliance of this policy, producing reports for the risk management committee (RMC) to review monthly. Every system deployed in any environment operated by Green Pace is expected to be in compliance with this policy at all times.

Staff members, consultants, or employees found in violation of this policy will be subject to disciplinary action, up to and including termination.

Exceptions Process

Any exception to the standards in this policy must be requested in writing with the following information:

- Business or technical rationale
- Risk impact analysis
- Risk mitigation analysis
- Plan to come into compliance
- Date for when the plan to come into compliance will be completed

Approval for any exception must be granted by chief information officer (CIO) and the chief information security officer (CISO) or their appointed delegates of officer level.

Exceptions will remain on file with the office of the CISO, which will administer and govern compliance.

Distribution

This policy is to be distributed to all Green Pace IT staff annually. All IT staff will need to certify acceptance and awareness of this policy annually.

Policy Change Control

This policy will be automatically reviewed annually, no later than 365 days from the last revision date. Further, it will be reviewed in response to regulatory or compliance changes, and on demand as determined by the OCISO.

Policy Version History

Version	Date	Description	Edited By	Approved By
1.0	08/05/2020	Initial Template	David Buksbaum	
[Insert text.]	10/11/2026	Completed security coding standards, risk assessments, automated detection tools, DevSecOps automation, encryption policies, and Triple-A controls.	Jose Alonso	Pending
[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]	[Insert text.]

Appendix A Lookups

Approved C/C++ Language Acronyms

Language	Acronym
C++	CPP
C	CLG
Java	JAV